Intermediate — stream the answer — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: a long model reply should type itself out on screen the way ChatGPT does, not appear all at once after a blank, silent 15-second wait. The web layer's job here is simple — it just forwards the tokens your chain already yields one by one, as they arrive. **If not:** every slow answer would look like a frozen, broken screen, even while the app is actually working perfectly underneath.

The token generator

In a real app this is your `chain.stream(...)`; here it's a stub that yields pieces so the web code is the focus:

```
# practice/pipeline.py
import time

def stream_answer(text: str):
    reply = f"You said '{text}'. Here is a longer, streamed answer."
    for word in reply.split():
        time.sleep(0.02)          # simulate the model thinking
        yield word + " "
```

Streamlit: `st.write_stream`

```
# practice/app_stream_streamlit.py   —   streamlit run app_stream_streamlit.py
import streamlit as st

from pipeline import stream_answer

st.title("Streaming Answer")
text = st.text_input("Ask something:")
if st.button("Answer") and text:
    st.write_stream(stream_answer(text))   # renders tokens as they arrive
```

`st.write_stream` takes a generator and renders each chunk the moment it's yielded. If your LCEL chain streams, pass `chain.stream(inputs)` straight in.

FastAPI: `StreamingResponse`

```
# practice/app_stream_fastapi.py   —   uvicorn app_stream_fastapi:app --reload
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel

from pipeline import stream_answer

app = FastAPI()

class AskRequest(BaseModel):
    text: str

@app.post("/stream")
def stream(body: AskRequest):
    return StreamingResponse(stream_answer(body.text),
                             media_type="text/plain")
```

`StreamingResponse` wraps the same generator; the browser (or any client) receives the body in chunks as they're produced, not all at once at the end.

Testing the stream (no server)

```
# practice/test_stream.py   —   python -m practice.test_stream
from fastapi.testclient import TestClient

from app_stream_fastapi import app

def test_streamed_body() -> None:
    client = TestClient(app)
    response = client.post("/stream", json={"text": "hello"})
    result = "OK"
    if response.status_code != 200:
        result = "WRONG"
    if "hello" not in response.text:
        result = "WRONG"
    print(f"case 1 (streamed answer arrives) -> {result}")

if __name__ == "__main__":
    test_streamed_body()
```

Expected: the response assembles to the full answer (which echoes "hello"), delivered in chunks — in a browser you'd watch it type out word by word.

The one rule: the pipeline yields tokens; the web layer only forwards them. Never collect the whole answer and then fake a stream — pass the generator through (`st.write_stream(gen)` / `StreamingResponse(gen, ...)`) so the user sees the first words immediately.